

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 함수 전달 방식 개념정리	date

2026.06.08

C++ 함수 전달 방식 개념정리

핵심

Call by Value, Call by Address, Call by Reference는 함수에 값을 넘기는 방식이다. 이 개념은 함수 안에서 원본 값을 바꿀 수 있는지, 복사 비용이 생기는지, 안전하게 데이터를 전달할 수 있는지를 판단하기 위해 필요하다.

왜 필요한가

함수는 값을 받아서 작업한다. 그런데 함수 안에서 값을 바꿨을 때 원본도 바뀌어야 하는 경우가 있고, 원본은 보호해야 하는 경우가 있다. 이 차이를 모르면 함수는 실행됐는데 main의 값이 그대로이거나, 반대로 원하지 않았는데 원본이 바뀌는 문제가 생긴다.

Call by Value

값을 복사해서 함수에 전달하는 방식이다.

```
void change(int x) {  
    x = 10;  
}  
  
int main() {  
    int num = 5;  
    change(num);  
    cout << num << endl;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과

`num`은 5 그대로이다.

왜 필요한가

Call by Value는 원본 값을 보호하기 위해 필요하다. 함수 안에서 실수로 값을 바꿔도 원본은 바뀌지 않는다. 작은 정수나 실수 값을 넘길 때는 간단하고 안전하다.

주의

구조체나 큰 객체를 값으로 넘기면 복사 비용이 커질 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 함수 전달 방식 개념정리	date

Call by Address

주소를 함수에 전달하는 방식이다. 포인터를 사용한다.

```
void change(int* x) {  
    *x = 10;  
}  
  
int main() {  
    int num = 5;  
    change(&num);  
    cout << num << endl;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 `num`이 10으로 바뀐다.

왜 필요한가

Call by Address는 함수 안에서 원본 값을 직접 바꾸기 위해 필요하다. 또한 큰 데이터의 주소만 넘기면 되기 때문에 복사 비용을 줄일 수 있다.

예시:

- 함수 안에서 두 값을 교환할 때
- 배열을 함수에 넘길 때
- 구조체 값을 수정할 때
- C에서 원본 값을 바꾸고 싶을 때

주의 포인터는 잘못된 주소를 사용하면 오류가 날 수 있다. `NULL` 포인터나 이미 해제된 메모리에 접근하지 않도록 조심해야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 함수 전달 방식 개념정리	date

Call by Reference

참조자를 통해 원본 변수에 다른 이름을 붙여 함수에 전달하는 방식이다.

```
void change(int& x) {  
    x = 10;  
}  
  
int main() {  
    int num = 5;  
    change(num);  
    cout << num << endl;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 `num`이 10으로 바뀐다.

왜 필요한가

Call by Reference는 포인터보다 문법이 간단하게 원본을 바꾸기 위해 필요하다. C++에서 자주 사용하며, 객체를 복사하지 않고 함수에 넘길 때도 유용하다.

주의 C에는 `int& x` 같은 참조자 문법이 없다. C에서는 원본을 바꾸려면 포인터를 사용하는 Call by Address를 사용한다.

const Reference

복사는 피하고 싶지만 원본은 바꾸고 싶지 않을 때 사용한다.

```
void printName(const string& name) {  
    cout << name << endl;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

큰 객체를 값으로 넘기면 복사 비용이 커진다. 그렇다고 참조로 넘기면 함수 안에서 원본이 바뀔 수 있다. `const reference`는 복사 비용을 줄이면서 원본을 보호하기 위해 필요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 함수 전달 방식 개념정리	date

비교

Value	원본 보호, 복사 발생
Address	원본 변경 가능, 포인터 사용
Reference	원본 변경 가능, 문법 간단
Const Reference	복사 비용 감소, 원본 보호

실전 선택 기준

작은 값이고 원본을 바꾸면 안 된다면 Call by Value를 사용한다.

원본 값을 바꿔야 하고 C 문법이라면 Call by Address를 사용한다.

C++에서 원본 값을 바꿔야 하고 문법을 간단히 쓰고 싶다면 Call by Reference를 사용한다.

큰 객체를 넘기지만 원본은 바꾸면 안 된다면 const Reference를 사용한다.

최종 정리

함수 전달 방식은 함수가 원본 값을 바꿀 수 있는지 결정하는 중요한 개념이다. Value는 안전하지만 복사 비용이 있고, Address와 Reference는 원본을 바꿀 수 있다. const Reference는 C++에서 큰 객체를 안전하고 효율적으로 넘기기 위해 필요하다.

확인 질문

- Call by Value에서 원본 값이 바뀌지 않는 이유는 무엇인가?
- 원본 값을 바꾸려면 어떤 방식을 써야 하는가?
- C와 C++에서 모두 사용할 수 있는 방식은 무엇인가?
- const Reference는 왜 필요한가?

어디에 사용하는가

함수 전달 방식은 함수가 원본 데이터를 바꿔야 하는지, 보호해야 하는지 결정할 때 사용한다.

Call by Value 사용 예: 원본 값을 바꾸면 안 되는 계산에 사용한다.

```
int square(int x) {  
    return x * x;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

'x'는 복사본이므로 원본 값은 안전하다.

Call by Address 사용 예: C에서 두 값을 바꾸는 swap 함수에 사용한다.

```
void swapValue(int* a, int* b) {
```

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 함수 전달 방식 개념정리	date

```
int temp = *a;
*a = *b;
*b = temp;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

호출:

```
swapValue(&x, &y);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Call by Reference 사용 예: C++에서 더 깔끔하게 swap을 만들 때 사용한다.

```
void swapValue(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

호출:

```
swapValue(x, y);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

const Reference 사용 예: 게임 캐릭터 정보나 책 정보처럼 큰 객체를 읽기만 할 때 사용한다.

```
void printBook(const Book& book) {
    cout << book.title << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

비유

Call by Value는 복사본을 주는 것이다. 친구가 복사본에 낙서해도 원본은 깨끗하다. Call by Address는 원본이 있는 집 주소를 알려주는 것이다. 친구가 그 집에 가서 원본을 직접 고친다. Call by Reference는 원본에 별명을 붙여주는 것이다. 별명으로 고쳐도 원본이 바뀐다.